From Trading Bot to Trading Agent: How to Build an AI-based Investment System

37 min read · Nov 1, 2025


Jung-Hua Liu Follow

Listen Share

Introduction

The rise of artificial intelligence (AI) is transforming algorithmic trading from rigid, pre-programmed bots into adaptive "trading agents" capable of understanding context and learning on the fly. Traditional trading bots — though fast and disciplined — often follow fixed rules that struggle to handle novel scenarios or sudden market regime changes. In contrast, new AI-driven trading agents leverage advanced models (especially Large Language Models, LLMs) to interpret a wealth of data (prices, news, social sentiment) and make more flexible decisions. Recent real-world trials highlight this paradigm shift. In the **Alpha Arena** live crypto trading competition, several state-of-the-art AI models (including OpenAI's GPT-5, Google's Gemini, Anthropic's Claude, Elon Musk's Grok, Alibaba's Qwen, and the specialized **DeepSeek** AI) were given $10,000 each to trade cryptocurrencies autonomously. The results were striking: *domain-focused* AI agents vastly outperformed generic bots. DeepSeek V3.1 achieved nearly **+40%** return in days, with xAI's Grok-4 close behind at +35%, while generalist models like GPT-5 and Gemini **lost** over 25%*[1]*. In fact, only two models (DeepSeek and Grok) were beating a simple buy-and-hold Bitcoin strategy during the contest*[2]*. This experiment underscored that adaptable AI agents with rich information intake can outshine fixed-rule systems. It is a clarion call for a new approach in trading system design.

This paper explores **how to build an AI-based investment system** that evolves a hard-coded trading bot into a **flexible trading agent** powered by AI. We discuss the limitations of traditional bots and how LLM-based models can overcome them by

reasoning over diverse data. We examine architecture choices (including multi-agent designs), integration of **DeFi/Web3** data sources, and techniques to maximize profit while managing risk. Real examples from cutting-edge AI trading competitions and open-source projects (such as FinGPT, CryptoTrade, etc.) illustrate the concepts. We also provide code snippets (in Python) demonstrating how to implement key components of an AI trading agent, from using LLMs for market analysis to deploying reinforcement learning for strategy optimization. Ultimately, we show an end-to-end roadmap for transitioning from static algorithms to intelligent agents that **learn, adapt, and perform** in dynamic markets.

Background: From Rule-based Bots to Intelligent Agents

Traditional Trading Bots: Early algorithmic trading systems, or "bots," operate on predefined rules or models. For example, a bot might be programmed to *"Buy Bitcoin if its 50-day moving average rises above the 200-day average"* or to execute arbitrage between exchanges. Such bots execute with superhuman speed and discipline, but they lack adaptability. They cannot deviate from their coding, even if market conditions change fundamentally. These systems are essentially **open-loop**: they take market price data as input and output trades based on static logic or trained predictors. While many bots use machine learning (e.g., predicting price direction from historical data), they typically **retrain only periodically** and don't incorporate unstructured information like news or qualitative signals. This rigidity makes them brittle. A sudden regulatory tweet or a macroeconomic shock can upend assumptions that the bot's strategy was based on, leading to losses or missed opportunities. In short, conventional bots excel at executing a fixed strategy but struggle with *contextual awareness* and *real-time learning*.

Rise of AI in Trading: Over the past decade, AI techniques have increasingly been applied to make trading systems more adaptive. Machine learning models have been used for price prediction, portfolio optimization, and risk management. Notably, **deep reinforcement learning (RL)** has shown promise: an RL agent learns by trial-and-error in a market environment, receiving rewards for profitable trades. For instance, researchers have trained crypto trading agents with on-chain data integrated into the state; one such system (CryptoRLPM) yielded **83% higher returns** than a Bitcoin buy-and-hold baseline and improved risk-adjusted performance (higher Sortino ratio)*[3][4]*. These AI-based approaches adapt to data patterns that static rules might miss. However, prior AI trading models largely focused on numerical *time-series data*. They acted as sophisticated signal processors, but often ignored the rich **context** that human traders consider (news headlines, social media sentiment, new protocol launches, etc.). This is where **Large Language Models** are making a profound impact.

Large Language Models (LLMs) — A Game Changer: LLMs like GPT-4, ChatGPT, Google's Gemini, etc., are AI models trained on vast text corpora, enabling them to understand and generate human-like language. While language may seem unrelated to trading, financial markets are heavily driven by **information flow** in news articles, tweets, research reports, and even chat forums. LLMs can read and interpret this unstructured data at scale, something traditional bots could never do. For example, OpenAI's ChatGPT can analyze financial news to summarize market sentiment and even interpret technical indicators from descriptions*[5]*. It can also

assist in **coding trading strategies** or explaining market concepts*[6]*. However, out-of-the-box ChatGPT has limitations: it doesn't have direct access to live data and needs integration with external feeds for real-time use*[7]*. Newer models aim to address such gaps. Elon Musk's **Grok**, developed by xAI, connects with the X (Twitter) platform to perform real-time sentiment analysis*[8]*. By monitoring social media chatter, Grok can gauge market mood and breaking trends — a crucial edge for short-term crypto trading. (Its downside is vulnerability to misinformation or manipulation in social data*[9]*, a risk we discuss later.) Google's **Gemini** is a multimodal LLM that can process not just text but images, audio, even video*[10]*. In trading, this could mean analyzing charts, reading video interviews of Fed officials, or scanning satellite images for commodity estimates. Gemini also has integration with Google's search, giving it real-time web access*[11]*. Meanwhile, specialized **fintech LLMs** are emerging. **DeepSeek** is one such model tailored for finance; it emphasizes efficient, cost-effective AI for traders, offering strong sentiment analysis and predictive analytics capabilities*[12]*. DeepSeek allows a degree of customization — users can fine-tune it to their specific strategies*[13]*. The proliferation of these models indicates that LLMs are injecting *contextual intelligence* into trading systems, bridging the gap between quantitative data and qualitative insight.

In summary, trading technology is evolving from mechanical bots to **cognitive agents**. Next, we define what it means to be a trading "agent" and how it differs from a traditional bot.

Trading Bot vs. Trading Agent

Defining a Trading Bot: A trading bot is an automated system with a fixed *policy* for how it trades. This policy could be a simple set of rules (if X and Y, then buy Z) or a more complex algorithm (like a neural network predicting prices). Critically, the bot's behaviour is *predetermined* by its programming or training. It does not truly *understand* context; it just reacts in a pre-set way to inputs. For example, a momentum trading bot might always buy after a 5% price rise in a day, regardless of *why* the price moved. Traditional bots also usually operate in isolation, focusing on market data alone. They lack memory beyond recent price history and do not autonomously seek new information. This often leads to suboptimal decisions when markets are driven by news outside the bot's purview. In volatile crypto markets, many bots have faltered during major news events (exchange hacks, regulatory bans, Elon Musk's tweets about Dogecoin, etc.) because such events don't fit the bot's historical price pattern training.

Defining a Trading Agent: A trading *agent*, in the AI sense, is an autonomous system that perceives its environment, makes decisions, and can adapt its strategy to achieve its goals (typically maximizing returns while managing risk). An agent is **interactive** and often stateful: it can gather new data, update its knowledge or strategy, and even "reason" about what to do next. Importantly, an agent can be built to understand *context and intent*. With LLMs, we can equip a trading agent with the ability to interpret *why* the market is moving, not just how. For example, an agent can detect that a sudden drop in ETH price is due to a smart contract exploit news, and decide differently than it would for a routine fluctuation. Trading agents often utilize **closed-loop feedback**: they observe outcomes of their actions and adjust future decisions accordingly (a kind of learning or reflection). This might be via on-line learning, or simply by dynamic re-planning using an up-to-date world model. In essence, a trading agent behaves more like a human trader — reacting not just to price ticks but to *events*, and learning from experience.

From Fixed Strategies to Adaptive Behavior: The difference between bot and agent is perhaps best illustrated by the recent AI trading competition mentioned earlier. Each participating AI was given the same objective (grow $10k capital) and same tools (access to a crypto trading platform), so one might expect similar performance if they were all just "bots." Instead, their performances diverged drastically, reflecting distinct **strategies and adaptability**. The top performers behaved like skilled human investors with well-defined styles:

- **DeepSeek V3.1** was described as a *"Patient Sniper"*, making only 6 trades over several days, mostly long positions held for ~21 hours each*[14]*. It waited for high-confidence opportunities and then let winners run. This indicates an ability to **filter noise** and act only on strong signals, a hallmark of an intelligent agent. DeepSeek likely incorporated a sophisticated internal model of the market; indeed its design benefits from the founder's quantitative fund background*[15][16]*. Despite criticisms in a U.S. report, DeepSeek's live performance validated its approach*[14]*.
- **Grok-4** (from xAI) acted as a *"Cautious Holder"*, making just 1 trade in the same period (holding a position ~54 hours)*[17]*. Grok's unique edge was its architecture that pulls in real-time network information (like social sentiment and news)*[17]*. By integrating these off-chain signals, Grok could confidently hold through volatility, exhibiting a long-term conviction that typical bots rarely

have. Its strategy shows **context-awareness** — it likely held because external sentiment/news supported the position, not just price momentum.

These two AI agents dramatically outperformed not just simple bots but even some advanced peers. By contrast, **Gemini 2.5 Pro** (Google's model) behaved like a high-frequency trader, executing 47 trades with average hold time <7 hours, and ended up with about **-30%** loss*[18]*. This rapid-fire strategy incurred high transaction costs and overtrading, suggesting a lack of adaptive risk management*[19]*. It's plausible that Gemini, being a generalist multimodal model, lacked fine-tuning for trading and so defaulted to an overly frenetic approach. Similarly, GPT-5 (a hypothetical OpenAI model in this test) lost ~26%, making 12 trades with ~23 hours hold on average*[20]*. GPT-5's strategy seemed misaligned with the market regime, perhaps due to deficiencies in its reward optimization or risk controls*[21]*. These weaker performances underscore that simply being a powerful model is not enough; without proper specialization or learning, an AI can flounder as a trader.

A striking observation from a follow-up analysis was that **each AI model had different strengths** across domains. DeepSeek, while excelling in trading, performed poorly in a subsequent poker AI tournament; Gemini was the opposite — a strong poker player but a weak crypto trader*[22]*. This reinforces a key point: intelligence is **domain-specific**. A trading agent benefits from knowledge and training specific to financial markets. LLMs endowed with general reasoning (like poker logic) might still fail at trading unless they can adapt to market-specific patterns and data. Therefore, building a successful trading agent requires combining *general intelligence* (reasoning, language understanding) with *domain expertise* (financial knowledge, specialized training) in a cohesive system.

In summary, a trading agent goes beyond executing a fixed strategy — it perceives context, adapts strategy, and learns from outcomes, much like a human trader but augmented with superhuman data processing. The next sections will delve into how Large Language Models can be harnessed to create such an agent, and the practical steps to implement an AI-based investment system.

The Role of LLMs in an AI Investment System

At the heart of our AI-based trading agent is the **Large Language Model**, which serves as the "brain" for understanding information and even making decisions. But LLMs are not magic black boxes — to use them effectively, we need to assign them well-defined roles within our system. Below we explore several crucial roles LLMs

(and their derivatives) can play, and provide code examples illustrating how to implement these functions.

1. LLM as Market Information Analyst

One of the most valuable uses of an LLM in trading is digesting and interpreting *unstructured data* — news articles, social media, announcements, forum discussions, even transcripts of speeches. These sources often contain clues to market direction (think of how the phrase "ETF approval" in a news headline can send Bitcoin soaring). A traditional bot that only sees price and volume would miss these signals. An LLM-based analyst can convert such text into actionable insight, such as a sentiment score or a summarized outlook.

Sentiment Analysis Example: Suppose our agent wants to gauge the market sentiment from news headlines and tweets. We can employ a specialized model like **FinBERT** (a BERT model fine-tuned for financial sentiment) or use a prompt with a general LLM to classify sentiment. Below is a code snippet using a Hugging Face transformer pipeline with a FinBERT model for sentiment analysis:

```
from transformers import pipeline

# Load a financial sentiment analysis model (FinBERT in this case)
classifier = pipeline("sentiment-analysis", model="ProsusAI/finbert")
# Example news headline
news_headline = "Bitcoin ETFs get greenlight: SEC approves first spot BTC ETF."
result = classifier(news_headline)[0]
print(f"Headline: {news_headline}\nSentiment: {result['label']} (confidence
{result['score']:.2f})")
```

In this snippet, ProsusAI/finbert is a public financial-sentiment model. Running this might output something like: *Sentiment: POSITIVE (confidence 0.95)* for the given bullish headline. Our trading agent could use this information to adjust its strategy (e.g. leaning long on BTC if news sentiment is strongly positive). In practice, one would aggregate many such headlines/tweets and perhaps take a weighted sentiment score.

LLM-based Analysis via Prompting: Alternatively, we can utilize a general LLM (like GPT-4 via API) to perform more complex analysis. For instance, not just *what* the

sentiment is, but *why* and *what might happen next*. Here's a conceptual example using OpenAI's API:

```
import openai
openai.api_key = "YOUR_API_KEY"
prompt = """You are an AI financial analyst.
News: "Ethereum network upgrade succeeds, analysts predict increased DeFi activity."
Tweet: "Major upgrade live on #Ethereum — bullish for $ETH and staking rewards!"
Question: Based on this news and tweet, what is the market sentiment and how might
ETH price react?"""
response = openai.Completion.create(
engine="gpt-4",
prompt=prompt,
max_tokens=150,
temperature=0.5
)
analysis = response.choices[0].text.strip()
print(analysis)
```

This prompt gives the LLM some context and asks for an analysis. A GPT-4 powered response might say: *"Both the news and tweet convey a bullish sentiment for Ethereum. The successful network upgrade and positive commentary likely increase investor confidence, potentially leading to a short-term price rally for ETH, especially in DeFi-related tokens. However, one should monitor trading volume and on-chain activity for confirmation."* This kind of nuanced insight — tying a technical event to market impact — is something an LLM can provide that a numeric model cannot. Our agent can feed such analysis into its decision logic.

Continuous News Monitoring: In an operational system, the agent would continuously fetch the latest news and social feeds. One might use APIs (e.g. Twitter API for tweets, RSS feeds for news sites, or specialized crypto news APIs) and then pass the content to the LLM for summarization or sentiment scoring. There are open-source projects like **FinGPT** that facilitate this. FinGPT is an open-source Financial LLM that emphasizes easy adaptation to new data*[23]*. It provides pipelines to automatically fetch internet-scale financial data and incorporate it into the model's knowledge. FinGPT's design addresses a key challenge: LLMs can get outdated quickly in finance. (For example, **BloombergGPT**, a 50-billion parameter model trained on financial data, would be expensive to retrain frequently; it took

~$3M to train over 53 days*[24]*.) FinGPT instead allows *lightweight fine-tuning* on recent data at much lower cost (hundreds of dollars), meaning the LLM-based analyst can stay up-to-date with the latest market regime*[23]*. FinGPT also highlights the use of **Reinforcement Learning from Human Feedback (RLHF)** to fine-tune an LLM to user preferences (e.g. risk aversion or investment style)*[25]*, an approach that can make our agent's analysis more personalized and aligned with its user's goals.

In summary, using an LLM as a market analyst, our agent gains a **real-time pulse of market sentiment and knowledge**. This goes far beyond any fixed keyword-based news alert that older bots used. It allows the agent to answer: *"What is happening and why might it matter for asset prices?"* — which is a foundational ability for a truly intelligent trading system.

2. LLM as Strategy Generator and Reasoner

Another role for LLMs is as a **strategy brain** — generating or selecting strategies and reasoning about the best action to take. Traditional bots have one hardwired strategy, but an AI agent could dynamically switch strategies or even come up with new ones on the fly. Large Language Models, especially when combined with a technique like *Chain-of-Thought prompting*, can reason through complex decisions, almost like an experienced trader thinking through a problem.

Dynamic Strategy Selection: An agent might maintain a library of known strategies (trend-following, mean-reversion, statistical arbitrage, etc.). An LLM can be tasked with choosing which strategy suits the current context. For example, if the market regime (deduced from analysis) is highly volatile with no clear trend, the LLM might suggest a mean-reversion or market-neutral strategy. If the LLM detects a strong bullish narrative in the news and positive technical momentum, it might suggest a trend-following strategy focusing on long positions. This can be done via prompting. For instance:

> *strategy_prompt = """You are an AI trading assistant.*
> *Market situation:*
> *— BTC price above 20-day average, strong upward momentum.*
> *— Macro news: Federal Reserve signals no more rate hikes.*
> *— Crypto sentiment: very positive (fear & greed index high).*
> *What trading strategy is appropriate? Choose from:*
> *(A) Trend-following long,*

```
(B) Mean-reversion short,
( c) Market-neutral arbitrage.
Explain your choice.""""
response = openai.Completion.create(engine="gpt-4", prompt=strategy_prompt,
max_tokens=100)
print(response.choices[0].text.strip())
```

Given the described bullish scenario, the LLM would likely pick **(A) Trend-following long** and explain something like: *"The combination of strong upward momentum and positive news/sentiment suggests a sustained uptrend, so a trend-following long strategy is appropriate to capitalize on further gains."* This explanation capability is valuable — it provides an *interpretable rationale* for the agent's choice, helping developers or users trust the agent's decision-making.

Generating Trade Ideas or Code: Beyond choosing from predefined strategies, an advanced agent could generate entirely new trading rules or even code for itself. LLMs like ChatGPT have been used by traders to prototype trading algorithms in plain English which the model then translates to code. For example, one could prompt: *"Write a Python function to execute a grid trading strategy on ETH/USD, with parameters X, Y, Z."* and the LLM will produce code. Our agent could conceivably modify its own strategy code via such prompts (though one must be cautious with self-modifying systems!). A safer approach is to have an LLM suggest parameter tweaks. E.g.: *"Volatility is rising, should I widen my stop-loss? If so, by how much?"* and use the answer to adjust the bot's settings.

Reasoning about Risk Management: A critical aspect of strategy is managing risk (position sizing, stop losses, take-profit levels). LLMs can help here by reasoning about scenarios. For instance, *"If Bitcoin suddenly drops 10% due to an ETF denial rumor, how should we adjust our portfolio exposure?"* The LLM might answer that scenario with advice to reduce leverage or hedge via options. While one shouldn't blindly trust an LLM with such decisions, its suggestions can be combined with quantitative checks. The benefit is the LLM can consider *qualitative logic*: e.g., "rumor-driven drop might reverse if rumor is false, but prudent to tighten stops."

In the Alpha Arena competition, we saw that some models likely had better risk management than others. GPT-5's poor performance was attributed to deficiencies in adapting to the environment and risk control*[20]*. An agent using an LLM for reasoning could explicitly analyze risk: *"Our portfolio is heavily long altcoins while the*

market is showing signs of risk-off (equities down, VIX up). This is risky." It could then suggest hedging or reducing positions. This kind of higher-level reasoning is something we can build into the agent by prompting the LLM to regularly perform a "portfolio risk review."

Code Example — Reasoning Loop: Consider a simplified loop where at each time step, after getting signals, we ask the LLM to output a final decision (buy/sell/hold) along with reasoning:

```
market_state = {
"BTC_price": 30000,
"BTC_trend": "up",
"news": "SEC delays decision on Ethereum ETF; market unsure.",
"sentiment": "mixed"
}
decision_prompt = f"""You are a trading agent.
Market state: {market_state}.
You have a long position in BTC.
Should you Buy, Sell, or Hold? Provide reasoning."""
reply = openai.Completion.create(engine="gpt-3.5-turbo", prompt=decision_prompt,
max_tokens=100)
print(reply.choices[0].text.strip())
```

The LLM might respond with something like: *"Hold. Rationale: BTC is in an uptrend, and while the ETF news is uncertain, no negative outcome is confirmed. Sentiment is mixed, not outright bearish. Maintaining the position with caution is prudent, possibly setting a stop-loss below $28k to manage downside risk."* The agent could parse this text: it sees the recommendation "Hold" and even extracts the suggested stop-loss level. Thu,s the LLM has not only given an action but also *contextualized it with reasoning and a risk measure*. This is far richer than a number output from a traditional predictive model.

In conclusion, using LLMs for strategy and reasoning turns a trading system from a rote executor into a **thoughtful planner**. It introduces flexibility: the agent can switch strategies or respond to novel situations in ways it wasn't explicitly programmed for, guided by the LLM's general knowledge and logical inference capabilities. This is a cornerstone of moving from a bot to an *adaptive agent*.

3. LLM-augmented Execution and Automation

Finally, an AI trading agent must execute trades and interact with financial systems (exchanges, wallets, smart contracts). Execution needs to be precise and fast. Here, traditional programming and infrastructure matter, but LLMs can still contribute, especially in complex environments like DeFi where interactions involve multiple steps or protocols.

Trade Execution via API: For a centralized exchange or broker, execution might be as simple as calling a REST API to place orders. This part doesn't require an LLM; in fact, you would *not* want to rely on an LLM to generate each API call (the overhead and unpredictability would add risk). Instead, you implement a straightforward execution module. For example:

```
import ccxt # a library for crypto exchange APIs
exchange = ccxt.binance({
"apiKey": "YOUR_API_KEY",
"secret": "YOUR_API_SECRET"
})
# If the decision from the agent is to buy 0.5 BTC at market:
order = exchange.create_market_buy_order("BTC/USDT", 0.5)
print(order)
```

This would execute a market buy on Binance for 0.5 BTC using ccxt (a popular open-source library that standardizes exchange API calls). The real intelligence lies in *when* and *what* to trade, which the LLM and other models determine; the execution module just carries it out efficiently.

DeFi and Web3 Execution: In decentralized finance, execution can be more complex. An agent might need to interact with smart contracts (for example, providing liquidity to a Uniswap pool, or borrowing on Aave). This involves constructing and signing blockchain transactions. While the low-level transaction formatting is best handled by web3 libraries (e.g., Web3.py for Ethereum), an LLM can be useful in planning the sequence of actions. For example, an *intent-based* DeFi investment agent might get a user intent like "maximize my stablecoin yield with low risk"*[26]*. The agent then needs to figure out an optimal strategy, which might involve multiple protocols (bridge assets to another chain, deposit into a lending platform, stake the yield token, etc.). A multi-step plan can be generated by an LLM given the current yields and opportunities. There is research in exactly this direction: Liu *et al.* (2023) describe a multi-agent system for personalized DeFi

strategies, where specialized agents (data, strategy, risk, execution) collaborate*[27] [28]*. One agent could query yield data across chains, another (with an LLM) could interpret the user's intent and devise a plan, and an execution agent would then perform transactions via account abstraction (gas management, etc.)*[29]*. The key advantage is **end-to-end automation**: the user says what they want, and the AI figures out how to do it, then does it. This is essentially an **autonomous DeFi agent**.

Example — Multi-step DeFi Action: Suppose our agent decides to implement a strategy: *swap ETH to DAI, then lend DAI on Compound*. We could have an LLM outline the steps and a web3 module execute them:

```
# Pseudocode for DeFi actions using web3 (assumes web3 is set up and contracts loaded)
strategy_steps = [
"Swap 5 ETH for DAI on Uniswap v3",
"Supply the obtained DAI to Compound to earn interest"
]
for step in strategy_steps:
print(f"Executing: {step}")
if "Swap" in step:
# call Uniswap router contract with swap parameters (precomputed via a DEX aggregator perhaps)
tx = uniswap_router.swap_exact_eth_for_tokens(...parameters...)
elif "Supply" in step:
# call Compound's cDAI contract to mint cDAI (supply DAI)
tx = cDAI_contract.mint(amount_of_DAI, {"from": agent_address})
# sign and send tx (using private key of agent)
web3.eth.send_raw_transaction(tx.sign(agent_private_key))
```

In this pseudocode, we hardcoded steps, but an LLM could generate the strategy_steps list given a high-level goal. There are initiatives like **DeFiChain's AI assistant** and **Fetch.ai's DeFi agent toolkit** that explore this space*[30]*. For safety, one would integrate on-chain data checks (make sure the expected output of a swap, check gas costs, etc.) rather than relying purely on the LLM's plan.

Collaboration of LLM with Traditional Modules: It's important to note that while LLMs are powerful, a robust trading agent uses them alongside traditional components. For execution, once an LLM (or the strategy module) decides what to do, the actual sending of orders or transactions is done by deterministic code. That

code should also enforce risk limits (e.g., "don't trade more than 10% of portfolio on a single position" or "if slippage > 1%, cancel the trade"). The LLM can *suggest* a trade, but the system around it will handle the precise execution and constraints.

In essence, the LLM acts as the "**thinking part**" of the agent — analyzing, strategizing, and sometimes even generating novel approaches — whereas the surrounding system handles the "**doing part**" — executing trades, monitoring positions in real-time, and making sure nothing crazy happens (like the LLM telling it to sell everything in a flash-crash panic — a human-inspired error we'd guard against!). This combination of *LLM-driven intelligence* with *traditional robust execution* creates a powerful, flexible yet safe trading agent.

System Architecture: Designing an End-to-End AI Trading Agent

Having covered the roles of an LLM, we now outline how to put everything together into an integrated system. A successful AI-based investment system will typically have a modular architecture, perhaps even a **multi-agent architecture**, where different components (or agents) handle different tasks and communicate. Below is a conceptual end-to-end design, incorporating both centralized exchange trading and DeFi capabilities (Web3), as well as learning mechanisms.

1. Data Ingestion and Processing: This layer gathers all relevant data and pre-processes it for the AI components. It includes:
— **Market Data Feeds:** Price, volume, order book from exchanges (via APIs or websocket). If focusing on crypto, this may include multiple exchanges or an aggregator for price indices. For on-chain markets (DeFi), it includes blockchain data like protocol states, transaction volumes, yields, etc. (One might use services like The Graph, or run nodes to get mempool and contract events.)
— **News and Social Data:** Feeds from news APIs (e.g. financial news, CryptoPanic API) and social media (Twitter/X, Reddit, Discord channels). This may involve using scrapers or third-party data services.
— **Feature Engineering:** Converting raw data into features for models. E.g., computing technical indicators (moving averages, RSI, etc.) from price data, aggregating sentiment scores from text (as demonstrated earlier), or extracting on-chain metrics (like number of active addresses, gas fees trends for Ethereum, etc.). Some features feed into LLM prompts (like the market_state we showed), others feed into numeric models.

This layer should be real-time or near-real-time. Low-latency is less critical than in pure HFT, since our agent's edge is more in *information processing* than millisecond speed. However, being up-to-date is crucial especially in crypto where news spreads quickly. Modern systems often use streaming architectures (Kafka, etc.) to pipe data continuously to AI modules.

2. AI Analysis and Decision Modules: The brain of the system, which can itself be subdivided:

— **LLM Analyst Agent:** As discussed, an agent (could be a process or module) that uses an LLM to analyze unstructured data and produce insights. It might output a summary of sentiment, a list of current risk factors, or detect if certain keywords (e.g. "SEC", "hacked", "partnership") are trending unusually, which could signal market-moving events. This agent can run on a schedule (e.g., analyze news every hour) or be triggered by events (e.g., a big price move triggers it to explain possible causes by looking at latest news). Tools like FinGPT can be leveraged here to maintain an up-to-date LLM for finance*[23]*.

— **Quantitative Model/Predictor:** Many systems will include models that predict price movements or volatility. These could be machine learning models (like an LSTM network forecasting next-hour returns, or a tree-based model ranking assets to long/short), or even simpler statistical models. Some agents might use **deep reinforcement learning** here: treat the market as an environment and output an optimal action. For instance, an RL agent could take state features (technical indicators + sentiment scores) and decide how to allocate the portfolio. The CryptoTrade research (EMNLP 2024) is effectively such an agent but with an LLM in the loop; it combined on-chain data, off-chain news, and a reflective learning mechanism to guide daily trades, outperforming traditional strategies*[31][32]*. In our architecture, the quant model and the LLM can complement each other. The quant model might be faster and capture high-frequency signals, while the LLM captures low-frequency but high-impact information.

— **Strategy and Planning Agent:** This component decides the overall strategy and converts insights/predictions into concrete trade decisions. It can be implemented by an LLM (as we showed in code prompts for strategy selection) or a rule-based logic that takes inputs from both the LLM analyst and the quant model. A sophisticated design is a **multi-agent system** where multiple specialist agents vote or debate on an action. Recent studies show that a multi-agent, multi-modal approach (with specialized agents for technical analysis, fundamental/news analysis, etc.) outperformed single-agent models and even market indices*[33][34]*. For example,

one agent could be a "Technical Guru" (looking solely at price patterns), another a "News Guru" (LLM-based), and a "Risk Manager" agent that oversees exposure. They could share their findings and come to a consensus or weighted decision. In practical terms, this might be an ensemble of models where the final decision is some function of all outputs. The design could be as simple as: *if both the quant model and LLM agree on a direction, take the trade; if they conflict, perhaps stay out or hedge.* Or one could implement a more explicit communication: e.g., let the LLM read the quant model's output and provide a final decision with justification (simulating an internal dialogue).

In any case, the output of this layer is a **set of trade instructions**: e.g., "Buy 2 ETH and sell 0.1 BTC" or "Rebalance portfolio to 60% crypto, 40% cash" or even "deploy capital to yield farming because trading opportunities are thin." The decision module also sets risk parameters for execution (position sizes, stop-loss levels, leverage, etc.).

3. Execution and Portfolio Management: This is the action layer that implements the trades and manages the portfolio:
— **Trade Execution Engine:** Responsible for sending orders to exchanges or transactions to blockchain. It should handle order books smartly (e.g., avoid excessive slippage by breaking orders if large, using limit orders if appropriate, etc.). If connected to multiple venues, it might route orders to where liquidity is best. For DeFi, it may interact with a wallet and smart contracts. This engine should also log executions, handle errors (retries, fallback exchanges), and confirm that orders are filled. In a live system, real money is on the line, so this part must be robust and tested extensively in simulation before going live.
— **Portfolio Tracking:** After trades, the agent's state (portfolio) must update. This includes available cash balance, current holdings of each asset, and perhaps unrealized P&L. A sub-module can continuously compute portfolio metrics (allocation percentages, risk measures like value-at-risk, etc.). These metrics may feed back into the decision module as state input. For example, if the portfolio breaches a risk threshold (say more than 50% in a single asset), the risk manager agent might flag this and trigger a rebalancing decision.
— **Learning/Reflection Module:** What sets an AI agent apart is that it can learn from outcomes. This module evaluates how the decisions turned out. If a trade was made based on an LLM's suggestion and it resulted in a big loss, the agent can analyze why. Perhaps the LLM was overly optimistic about a news piece — the reflection module could adjust the sentiment scoring process (maybe down-weight single-

source news). Or in an RL context, the reward from the market (profit or loss) is fed back to update the policy network. The CryptoTrade agent, for example, includes a reflective mechanism to refine its trading decisions by analyzing prior outcomes*[35]*. This is akin to a daily post-mortem: *"Yesterday, I lost money on SOL because I ignored a bearish divergence. Next time, pay more attention to technical signals when sentiment is only mildly positive."* An LLM can even be used to generate these reflective insights in natural language, which can then be coded into adjustments. Over time, this learning loop can significantly improve performance, as the agent avoids repeating mistakes.

To ground this architecture, consider an **end-to-end flow** on a given day: Morning data arrives — prices, some news of a hack on a DeFi protocol. The data module updates technical indicators and flags the news. The LLM analyst summarizes: *"News of a hack on XYZ protocol, causing broad negative sentiment in DeFi sector."* The quant model notes that all DeFi-related tokens are down 10%. The strategy agent (LLM) reasons that this might be an overreaction and a mean-reversion long on strong tokens (like ETH) could be profitable, but it also notes risk is higher. The risk agent says overall market volatility is up, so reduce position sizes. The final decision: it maybe signals to buy a small amount of ETH and perhaps short a weaker DeFi token as a hedge (market-neutral pair trade). The execution engine places these trades on the exchange. Throughout the day, the portfolio manager watches the P&L; if losses hit a stop threshold, it might trigger an automatic exit (risk protection). At day's end, the reflection module sees the hedge was not needed (the hack impact faded and everything bounced). It concludes that next time maybe a simpler long would suffice, but acknowledges the caution didn't hurt much. It records this as experience. Over many such cycles, the agent calibrates its responses.

This architecture, while complex, ensures **end-to-end functionality**: data → analysis → decision → action → learning, which is the hallmark of an intelligent trading agent. In implementation, one could use a combination of frameworks: for example, **LangChain** or custom code for orchestrating LLM calls and multi-agent dialogues, **stable-baselines3** or similar libraries for reinforcement learning components, and standard libraries for trading and web3 interactions. The system should also include **evaluation tools** — you'd backtest it on historical data, paper trade it in live markets before deploying real capital, and continue to monitor its performance and adjust as needed.

Implementation in Python: Illustrative Examples

To make the discussion more concrete, in this section, we provide simplified Python code snippets that demonstrate how pieces of the system can be implemented. These are illustrative and omit many real-world checks and complexities, but they serve as a starting point for a practitioner.

Example 1: Integrating LLM Analysis with Trading Signals

Imagine we have a basic trading signal from a quantitative model (e.g., an indicator that is +1 for buy, -1 for sell). We want to combine this with an LLM's sentiment analysis before making a trade. We will: (a) get the quant signal, (b) get an LLM-based sentiment, and © decide trade action based on both.

```
# Step (a): Quant signal (e.g., a simple momentum indicator for BTC)
price_history = get_price_history("BTC", days=5) # pseudo-function to get last 5 days prices
momentum = 1 if price_history[-1] > price_history[0] else -1 # +1 if price increased over 5 days, else -1

# Step (b): LLM sentiment analysis (could be via OpenAI or a local model)
latest_news = fetch_latest_news("BTC") # pseudo-function to get latest BTC-related news headline
sentiment = classifier(latest_news)[0]['label'] # using FinBERT as classifier from previous example
sent_score = 1 if sentiment == "POSITIVE" else -1 # simplistic conversion of label to +1/-1

print(f"Quant signal: {momentum}, News sentiment: {sentiment} -> {sent_score}")
```

Suppose momentum = +1 (market trending up) but the news sentiment came out NEGATIVE (maybe news of a regulation threat). The print might show: Quant signal: 1, News sentiment: NEGATIVE -> -1. *Now combine:*

```
# Step ( c): Decision logic combining signals
if momentum == 1 and sent_score == 1:
action = “Buy”
elif momentum == -1 and sent_score == -1:
action = “Sell”
else:
action = “Hold” # conflict between technical and sentiment signals
print(f”Decision: {action}”)
```

In our hypothetical scenario (momentum up, sentiment down), this would decide to “Hold” (no trade) because the signals conflict. In contrast, if both were positive, it would decide to Buy. This simple logic is a form of **signal fusion**, where the LLM’s output (sentiment) is merged with a quant signal. One could weight them or use a more complex rule (e.g., if sentiment is strongly negative, maybe even override a weak positive technical signal). The principle shown is how to incorporate LLM-derived insight into a trading strategy programmatically.

Example 2: Reinforcement Learning Agent for Crypto

Now let’s demonstrate how one might set up a reinforcement learning trading agent in Python using a library. We will use a pseudo environment for brevity, but conceptually, one could use **OpenAI Gym** style interface or **FinRL** (Financial Reinforcement Learning library). For example, FinRL provides environments for stock and crypto trading where state includes technical indicators and the agent learns to allocate portfolio.

Below, we outline using **Stable-Baselines3** (a popular RL library) with a custom trading environment:

```
import gym
from stable_baselines3 import PPO

# Assume we have a custom environment that follows gym interface
env = gym.make(‘TradingEnv-v0’) # this env could include price, indicators, etc.
model = PPO(“MlpPolicy”, env, verbose=1)
```

```
# Train the RL agent for some iterations
model.learn(total_timesteps=100000) # train on historical data in the environment

# After training, test the agent
obs = env.reset()
for _ in range(10):
action, _states = model.predict(obs)
obs, reward, done, info = env.step(action)
if done:
break
env.render()
```

In this snippet, TradingEnv-v0 would be a user-defined environment. Each step could represent one day or one hour of trading, with action maybe being a discrete choice (0 = hold, 1 = buy, 2 = sell) or continuous (like % of capital to allocate). We use PPO (Proximal Policy Optimization) as the RL algorithm. The agent is trained on historical data (e.g., past 2 years of BTC prices). After training, we simulate 10 steps to see what it does, and env.render() would show performance (maybe a portfolio value plot).

To integrate LLMs into this, there are two approaches: — *Provide LLM-based features to the RL agent.* For instance, include a sentiment score in the environment's state so the agent learns its relevance. This way, the LLM's analysis guides the RL decisions indirectly.
— *Use the LLM as the agent's policy.* This is experimental, but researchers are exploring it. It would mean at each step, instead of a neural network taking the state to output an action, we prompt an LLM with the state description and it outputs an action. The CryptoTrade project essentially did this using GPT-4 as a policy that is *reflectively* improved*[35]*. Reinforcement learning could fine-tune an LLM's responses via reward signals (a form of RLHF where "human feedback" is replaced by "market reward").

Given the complexity, the first approach (LLM-derived features) is more straightforward now. The code above can be extended by modifying the environment such that obs includes things like obs['sentiment'] = sent_score and obs['news_topic'] = … etc., in addition to price data. The neural network policy then

has those extra inputs. There is evidence this helps: an RL agent trained with on-chain and news data outperformed those without*[31]*.

Example 3: Multi-Agent Collaboration (Advanced)

To illustrate a multi-agent setup, let's consider splitting tasks between two agents in code: one agent (Agent A) focuses on **technical trading**, and another agent (Agent B) focuses on **news analysis**. They will communicate their suggestions and a coordinator will decide the final action.

We can simulate Agent A with a simple technical rule and Agent B with an LLM call, then combine:

```
# Agent A: Technical trader suggestion (e.g., based on moving average crossover)
fast_ma = sum(price_history[-5:]) / 5
slow_ma = sum(price_history[-20:]) / 20
agentA_suggestion = "Buy" if fast_ma > slow_ma else "Sell"

# Agent B: News analyst suggestion (using LLM sentiment from before)
agentB_suggestion = "Buy" if sent_score == 1 else "Sell"

print(f"AgentA suggests: {agentA_suggestion}, AgentB suggests: {agentB_suggestion}")
# Coordinator logic:
if agentA_suggestion == agentB_suggestion:
final_action = agentA_suggestion # both agree
else:
final_action = "Hold" # disagreement -> no trade (or could trust one agent over the other)
print(f"Final action: {final_action}")
```

This is a simplistic coordination mechanism (requiring consensus). More sophisticated approaches could assign weights or let one agent convince the other. For example, one could prompt an LLM to act as a referee: *"Agent A says BUY due to technicals. Agent B says SELL due to news. Who is likely correct? Justify."* The LLM might then output a reasoning balancing both arguments, effectively performing ensemble reasoning. The medium article we cited notes how agents can adjust each other's predictions via communication*[36]*. Implementing that might involve having Agent A and B output not just a direction but confidence levels and reasons, which a third process evaluates.

While multi-agent setups are more complex to code, frameworks are emerging. The **TradingAgents** framework (as per DigitalOcean's reference) and **FinRobot** aim to simulate multiple LLM agents debating trades*[37][38]*. Using such frameworks is beyond this paper's scope, but the takeaway is that splitting the problem among specialized agents can yield better performance by leveraging different strengths, much like a team of experts vs. a single generalist.

Example 4: Using FinGPT for Rapid Adaptation

We have discussed FinGPT conceptually; to give a flavor of its usage, consider that you want to fine-tune a small open-source LLM on recent financial news to personalize it for your needs (say you want it to better understand crypto jargon or a particular trading style). FinGPT provides tools to do instruction tuning on financial data*[39]*. Here's a pseudo-sequence of steps one might take with FinGPT (not actual code due to complexity):

```
# Pseudocode for fine-tuning an LLM with FinGPT utilities
from fingpt import DataCrawler, Finetuner

# 1. Use FinGPT DataCrawler to fetch recent crypto news headlines for last 3 months
crawler = DataCrawler(data_source="news", query="crypto OR bitcoin", start="2025–08–01", end="2025–11–01")
news_data = crawler.fetch()

# 2. Prepare instruction data for tuning (e.g., ask the model to summarize or sentiment-tag the news)
training_data = []
for item in news_data:
prompt = f"<News>{item['headline']}</News>\nIs this news good or bad for crypto markets? Explain."
response = human_label_or_existing_model(item) # assume we have labels or a rule to generate answers
training_data.append((prompt, response))

# 3. Fine-tune a base LLM (say Llama-7B) on this QA dataset
finetuner = Finetuner(base_model="meta-llama/7B", data=training_data)
finetuned_model = finetuner.train(output_dir="./finetuned_model", epochs=1)
```

This pseudo-code uses FinGPT-like components to gather data and perform **instruction fine-tuning**. The idea is that after this, finetuned_model would be better at understanding crypto news and responding in a way useful to trading decisions (perhaps more concise, or more attuned to market implications). FinGPT research showed that such fine-tuned models can outperform even GPT-4 on domain-specific tasks like financial sentiment*[40]*. If one doesn't have the capacity to train large models, FinGPT also makes use of parameter-efficient techniques like LoRA (Low-Rank Adaptation) to fine-tune a big model with relatively low compute*[41]*. The resulting model can be plugged back into our agent as the LLM analyst, ensuring it stays up-to-date and domain-specialized.

The above examples scratch the surface, but they reflect key implementation steps: combining signals, training an agent with RL, structuring multi-agent logic, and customizing an LLM. By assembling these pieces appropriately, one can create a prototype AI trading agent. In practice, substantial additional work is needed to handle edge cases, ensure reliability, and optimize performance. However, with modern libraries and cloud resources, an individual or small team can build a functional AI trading system — something that would have been infeasible just a few years ago.

Challenges and Considerations

Building and deploying an AI-based investment system is not without challenges. As we push from a simple bot to a sophisticated agent, we must be mindful of potential pitfalls and design considerations:

1. Data Quality and Latency: An AI agent is only as good as the data it sees. Incorporating news and social feeds means dealing with noisy, sometimes false information. Misinformation can lead the agent astray (as noted, Grok can be misled by social media rumors*[9]*). We should implement filters or credibility scoring (perhaps giving more weight to trusted news sources than random tweets). Additionally, if data ingestion lags (say our news feed is minutes behind real-time), the agent might react too late. Ensuring low-latency feeds, especially for critical data, is important. In DeFi, on-chain data latency is another factor — if our agent doesn't see a protocol exploit until after funds are drained, it's too late to react. Running our own blockchain nodes or using push notification services for on-chain events can mitigate this.

2. Model Limitations and Updates: Pre-trained LLMs have knowledge cut-off dates and may lack specific financial context. We addressed this with fine-tuning (FinGPT approach) and giving real-time data via prompts or tools. It's crucial to continuously update the LLM or its information. An outdated model might, for instance, not know about a new kind of crypto instrument (like liquid staking tokens) and misinterpret related news. Regular model updates or prompt engineering can keep it relevant. Also, while powerful, LLMs are **probabilistic** and can produce erroneous or inconsistent outputs (hallucinations). Rigorous testing is needed to ensure the agent doesn't act on a hallucinated insight. For example, one could cross-verify critical LLM outputs: if the LLM says "Exchange X is hacked!" the agent should verify via another source/API before panic-selling.

3. Risk Management Must Be Enforced: An AI agent, especially one learning or reasoning freely, might occasionally propose risky actions (like putting 100% of capital into a single volatile token, or using maximum leverage because it 'thinks' a trade is a sure win). Human traders have instincts and regulatory limits to check such behavior; we must imbue the agent with similar guardrails. Hard constraints (like position size limits, leverage caps, stop-loss mandates) should be built into the execution layer. Moreover, a dedicated risk management module (possibly a separate agent) should continuously monitor exposure. As seen in the Alpha Arena, models lacking good risk control (GPT-5, Gemini) ended up with big losses*[20][18],* whereas the top models implicitly controlled risk via low frequency, high-conviction trading*[14][17]*. We can take inspiration from DeepSeek's "patient sniper" approach — trade less often, only when confidence is high, and avoid over-trading. Our agent should quantify its confidence (LLM can output a confidence score, or an ensemble disagreement can signal uncertainty) and only take large bets when multiple signals align strongly.

4. Explainability and Debugging: AI decisions can be opaque. If the agent loses money on a trade, we need to understand why to improve it. Logging the chain-of-thought from the LLM (the reasoning it gave) and the signals that led to a decision is very helpful. This way, when reviewing a bad trade, one might find "The LLM misinterpreted the news" or "The RL model overemphasized a short-term price dip." Efforts in explainable AI, such as forcing the LLM to provide justifications, pay off here. In financial settings, explainability is also important for user trust and possibly compliance. If this system were used in a fund, regulators or risk officers might demand explanations for large moves. The fact that our design encourages

the agent to articulate reasoning (e.g., "Trend is strong, news is good, so I buy") is a feature, not just a debugging aid.

5. Computational and Cost Aspects: Running LLMs, especially large ones like GPT-4, can be expensive and slow. There is a trade-off between intelligence and speed. In trading, certain opportunities require split-second reactions (e.g., arbitraging a price difference). Our LLM-based agent is likely not suitable for such ultra high-frequency trades due to inference latency of the model. Instead, it should focus on low to medium frequency decisions where a delay of a few seconds (or even minutes) is acceptable. We can optimize by using smaller local models for routine tasks and reserving big model calls for major decisions or analyses that truly need them. Open-source models fine-tuned to our needs could run on our own GPUs to reduce API costs. The FinGPT project, for example, is all about using open models to avoid reliance on closed APIs*[42][43]*. That said, if using cloud services (OpenAI, etc.), one must budget for API usage. Also, the infrastructure to gather and store all these data streams can be non-trivial (databases for price history, news archives for model training, etc.).

6. Security and Robustness: In a DeFi context, an agent controlling private keys and executing blockchain transactions must be extremely secure. Any vulnerability could be catastrophic (imagine an exploit where someone prompts the LLM with a malicious instruction to send funds to an attacker's address — the agent must be restricted from doing such obviously harmful things). Isolating the execution permissions, using multi-signature or requiring human confirmation for very large transfers, are measures to consider. Robustness also means handling unexpected events: what if the LLM API is down or returns an error? The agent should have fallback behaviors (maybe revert to a simpler strategy temporarily). What if market conditions are unlike anything seen before (black swan event)? The agent might need to enter a capital preservation mode (e.g., go mostly to cash) rather than try to trade through highly uncertain conditions. These are analogous to how a seasoned human trader would cut risk during extreme turbulence.

7. Ethical and Regulatory Considerations: While not a technical challenge, it's worth noting that as AI agents trade more actively, regulators may scrutinize them. Issues like market manipulation could arise if, say, an LLM-based agent learns it can nudge sentiment by posting content online (a hypothetical but not impossible scenario if connected to execution). Ensuring the agent follows market rules and does not engage in unethical behavior is important. One should also be careful

deploying such a system in live markets without thorough testing — unintended consequences (flash crashes, liquidity strains) could occur if many agents behave similarly. The Alpha Arena experiment is encouraging in that it publicly *evaluated* AIs in real markets to see their behavior*[44],* setting a precedent for transparent testing of AI traders. We should continue in that spirit: test agents in sandbox or small-scale live settings, learn from mistakes, and improve them before scaling up.

In summary, moving to an AI-driven trading agent brings new challenges, but none are insurmountable. With careful design — combining AI with rule-based safeguards — and continuous monitoring, one can harness the power of LLMs and other models while mitigating risks. The key is to treat the trading agent as a **constant work-in-progress**: much like a human trader grows wiser with experience, our AI agent will evolve, and we must evolve its safeguards and training alongside it.

Conclusion

The evolution from a basic trading bot to a sophisticated trading agent represents a significant milestone in the intersection of AI and finance. In this paper, we detailed how integrating Large Language Models and their derivatives into trading systems enables a level of flexibility and context-awareness previously unattainable. An AI-based investment system can process **all forms of data** — numeric price feeds, textual news, on-chain metrics — and reason about them to make informed decisions. It shifts the paradigm from following fixed algorithms to **learning and adapting** in real time. We saw this vividly in the competition where specialized AI agents like DeepSeek and Grok outperformed more general models by wide margins*[1][14],* underscoring the value of domain-specific intelligence and strategy. By referencing open-source projects (FinGPT, CryptoTrade) and contemporary research, we illustrated practical pathways to build such systems, including code examples for key components.

The journey from bot to agent also mirrors a broader trend: financial AI systems are moving from *automation* to *autonomy*. Early bots automated execution of human-designed strategies; modern agents can autonomously devise and tweak strategies, essentially **investing on their own** within given mandates. This has profound implications. On the positive side, it can democratize advanced trading — an individual with a well-designed AI agent can compete in markets more effectively, as the agent encapsulates expertise that would otherwise take years to develop. It could also improve market efficiency by arbitraging away anomalies faster and incorporating information into prices more quickly (as AI agents digest news

immediately). On the flip side, the use of autonomous agents demands responsibility. We must ensure these agents act in ways that are aligned with human values and financial stability (avoiding cascade failures, for instance).

For crypto markets and DeFi in particular, AI agents might become indispensable. The complexity and speed of Web3, where thousands of tokens and protocols interact, are beyond any single human's capacity to monitor. An AI agent, however, can tirelessly scan across chains, read every governance proposal, track whale movements, and execute intricate yield strategies — effectively being a **24/7 portfolio manager** that never sleeps. As one piece of research noted, multi-agent AI frameworks in DeFi can handle cross-chain tasks and personalize strategies, potentially outperforming both static models and human teams*[33][34]*. The future might see swarms of such agents optimizing liquidity, making markets, and managing investments, all communicating in natural language with their human operators (or with each other).

In conclusion, transforming a trading bot into an AI-driven trading agent is a multifaceted endeavor, blending data engineering, machine learning, LLM technologies, and sound financial principles. By following an end-to-end design — from data ingestion to decision-making and execution — and by leveraging the power of LLMs for understanding and reasoning, one can build an investment system that is *greater than the sum of its parts*. This agent will not only maximize profit opportunities with its expanded intelligence but also adapt to the ever-changing landscape of markets. As our case studies and code snippets suggest, the tools to create such systems are increasingly accessible. With careful implementation and oversight, AI trading agents could very well become the norm in the next generation of financial markets, operating with a level of autonomy and insight that bridges the gap between human intuition and machine precision — truly the best of both worlds.

References:

· Alpha Arena AI Trading Competition results and analysis*[1][14][18][2][22]*.

· Binance Research on AI models (ChatGPT, Grok, Gemini, DeepSeek) for trading*[5][7][8][45][10][46]*.

· FinGPT open-source financial LLM project (AI4Finance Foundation)*[47][25][40]*.

· CryptoTrade reflective LLM-based trading agent (EMNLP 2024)*[31][32]*.

· Liu et al. on multi-agent AI architecture for personalized DeFi strategies*[3][4][33][34]*.

· BlockTempo/Foresight News coverage on AI trading and poker experiments*[2][22]*.

[1] [14] [17] [18] [19] [20] [21] [44] AI Large Model Real Trading Showdown: DeepSeek and Grok Lead the Way, Revealing Different Models' Investment Philosophies — ChainCatcher

https://www.chaincatcher.com/en/article/2213902

[2] [22] 炒幣還沒分勝負，AI 們又湊了桌德撲 | 動區動趨-最具影響力的區塊鏈新聞媒體

https://www.blocktempo.com/ai-poker-tournament-deepseek-gemini-grok-battle/

[3] [4] [26] [27] [28] [29] [33] [34] [36] Multi-Agent AI Architecture for Personalized DeFi Investment Strategies | by Jung-Hua Liu | Medium

https://medium.com/@gwrx2005/multi-agent-ai-architecture-for-personalized-defi-investment-strategies-c81c1b9de20c

[5] [6] [7] [8] [9] [10] [11] [12] [13] [45] [46] Grok vs. ChatGPT vs. Gemini vs. DeepSeek: Which AI is Best for Crypto Trading? | Abir Ama Ab on Binance Square

https://www.binance.com/en/square/post/22089175904882

[15] [16] DeepSeek Beats GPT-5, Gemini in Live Crypto Trading Battle | by Damon | Oct, 2025 | Medium

https://damoncote.medium.com/deepseek-beats-gpt-5-gemini-in-live-crypto-trading-battle-7531d31c5d8c

[23] [24] [25] [39] [40] [41] [42] [43] [47] GitHub — AI4Finance-Foundation/FinGPT: FinGPT: Open-Source Financial Large Language Models! Revolutionize We release the trained model on HuggingFace.

https://github.com/AI4Finance-Foundation/FinGPT

[30] DeFi AI Agent Starter Guide | Innovation Lab Resources

https://innovationlab.fetch.ai/resources/docs/next/examples/asione/asi-defi-ai-agent

[31] *[32]* *[35]* GitHub — Xtra-Computing/CryptoTrade: [EMNLP 2024] CryptoTrade: A Reflective LLM-based Agent to Guide Zero-shot Cryptocurrency Trading https://aclanthology.org/2024.emnlp-main.63.pdf

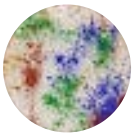
https://github.com/Xtra-Computing/CryptoTrade

[37] Your Guide to the TradingAgents Multi-Agent LLM Framework

https://www.digitalocean.com/resources/articles/tradingagents-llm-framework

[38] FinRobot: An Open-Source AI Agent Platform for Financial Analysis ...

https://github.com/AI4Finance-Foundation/FinRobot



Follow

Written by Jung-Hua Liu

562 followers · 11 following